

11 — Hardware Prefetching e Memory-Level Parallelism

Pré-requisito: [05-latency-vs-bandwidth.md](#)

Este arquivo formaliza algo que você já descobriu **na prática**, sem querer, no [examples/01-latency-benchmark/](#) deste módulo. A primeira versão daquele benchmark usava um stride fixo — e os resultados saíram planos, sem os degraus esperados entre níveis de cache. A correção foi trocar pra uma permutação verdadeiramente aleatória (algoritmo de Sattolo). Este arquivo explica **por que** isso foi necessário.

1. O que realmente aconteceu no seu benchmark

```
// versão original, com bug conceitual (não erro de sintaxe):  
size_t stride = 16;  
// ... next = (idx + stride) % n ...
```

Esse padrão — sempre pular exatamente **stride** posições — é **previsível**. E hardware moderno tem um circuito dedicado, o **prefetcher de hardware**, cujo único trabalho é detectar padrões previsíveis de acesso à memória e **buscar dado adiantado**, antes de você pedir. Um stride fixo é exatamente o padrão mais fácil de detectar que existe — o prefetcher "aprendeu" o padrão nas primeiras requisições e passou a buscar o resto com antecedência, escondendo a latência real que o benchmark queria medir. É por isso que a primeira versão saiu "rápida demais", sem os degraus.

2. O que é o prefetcher, mecanicamente

O prefetcher observa a sequência de endereços que o programa pede, tenta detectar um padrão (o mais simples: "sempre soma X ao endereço anterior"), e, se confiante no padrão, começa a buscar endereços **futuros** especulativamente — antes que o programa realmente os peça. Se a especulação está certa (o programa realmente pede aquele endereço em seguida), a latência daquele acesso já foi "paga" adiantada, escondida atrás do trabalho que o programa já estava fazendo. Se está errada, o trabalho de busca foi desperdiçado (mas não quebra nada — só gasta banda de memória à toa).

3. Por que isso importa: existem DUAS formas diferentes de "sofrer com memória"

Esse é o ponto central do arquivo. Não é "toda memória lenta é lenta do mesmo jeito" — os dois padrões abaixo, embora ambos envolvam "acessar memória", têm comportamento radicalmente diferente:

Padrão de dependência (o que o pointer chasing força)

```
load endereço A  
|  
v
```

```

descobre o endereço de B (só sabendo o QUE está em A)
|
v
load endereço B
|
v
descobre o endereço de C
|
v
load endereço C ...

```

Cada load **depende do resultado do load anterior** — você literalmente não sabe qual endereço pedir a seguir até ter o resultado do atual. Isso é chamado de **dependent load**. Não existe como sobrepor essas requisições (lembra do arquivo 05 — sobreposição exige saber de antemão o que pedir), e o prefetcher não tem como ajudar (ele não pode "adivinhar" o conteúdo de A antes de A ser lido). Você paga a latência cheia, uma requisição de cada vez. Isso é exatamente o que o pointer chasing do exemplo 01 força de propósito — pra medir latência real, sem trapaça de prefetch nem de sobreposição.

Padrão de streaming (o que percorrer um array faz)

```

load x[0]
load x[1]      <- o endereço já era conhecido de antemão, não depende do valor de
x[0]!
load x[2]
load x[3] ...

```

Aqui, o endereço do próximo acesso **não depende do resultado do acesso atual** — é só aritmética de índice, conhecida antes mesmo do primeiro load acontecer. Isso é chamado de **independent load** (ou access), e é exatamente o cenário onde tanto o prefetcher quanto o paralelismo natural de requisições em voo conseguem atuar.

4. Memory-Level Parallelism (MLP): quantas requisições "em voo" ao mesmo tempo

MLP é a métrica de quantas requisições de memória independentes um programa consegue ter pendentes (outstanding) simultaneamente — em progresso, ainda não retornadas. Hardware moderno tem uma fila limitada de requisições em voo (não é infinita), mas o número é maior que 1 — geralmente dezenas. Quanto mais alto o MLP que seu código consegue expor, mais perto do regime de bandwidth (arquivo 05) você chega, mesmo com latência individual ruim.

```

MLP baixo (dependent loads, tipo pointer chasing):
[load A]----->[load B]----->[load C]----->
tempo total ≈ 3 × latência individual

MLP alto (independent loads, tipo streaming):
[load A]----->
[load B]----->      <- todos em voo ao mesmo tempo

```

```
[load C]----->
tempo total ≈ 1 × latência individual (+ um pouco), não 3×
```

PERGUNTA RÁPIDA 1: por que a técnica de "múltiplos acumuladores independentes" do arquivo 02 (ILP) e o conceito de MLP aqui são, na prática, a mesma ideia aplicada em camadas diferentes do problema?

► Resposta

5. Conectando com Little's Law: concorrência NECESSÁRIA vs concorrência DISPONÍVEL

O arquivo 05 (seção 6-7) te deu uma fórmula pra calcular **quanta concorrência de requisição você precisa** pra saturar uma dada bandwidth, com uma dada latência — **concorrência necessária ≈ throughput × latência**. MLP, aqui, é uma pergunta diferente, e as duas juntas fecham o raciocínio completo:

Little's Law responde:

"Quanta concorrência eu PRECISARIA ter, pra saturar a bandwidth disponível?"
→ um número que depende só do HARDWARE (throughput-alvo e latência do hardware)

MLP responde:

"Quanta concorrência o meu PADRÃO DE ACESSO consegue realmente expor?"
→ um número que depende do CÓDIGO (dependent vs independent load)

O resultado real depende do menor dos dois. Um exemplo concreto, juntando os dois:

Little's Law diz: preciso de 20 requisições em voo pra saturar a bandwidth.

Mas o código é uma cadeia de pointer chasing (dependent load) –
MLP exposto pelo padrão de acesso: apenas 1 requisição em voo por vez.

Resultado: mesmo o hardware sendo capaz de sustentar 20 requisições simultâneas, o CÓDIGO nunca expõe mais que 1 – a latência NUNCA é escondida, não importa quão boa seja a bandwidth teoricamente disponível.

Essa é a razão exata de pointer chasing ser tão mais lento que streaming, mesmo em hardware idêntico: não é que a memória fique "mais lenta" pra pointer chasing — é que o padrão de acesso simplesmente não consegue gerar a concorrência que Little's Law diz ser necessária pra aproveitar a bandwidth que o hardware ofereceria de bom grado.

PERGUNTA RÁPIDA 2: se Little's Law calcula que você precisa de 39 requisições em voo (o exemplo do arquivo 05) pra saturar a bandwidth, mas seu hardware só consegue sustentar fisicamente 32 requisições em voo ao mesmo tempo (um limite real de fila, não do seu código), o que isso significa?

► Resposta

6. Por que isso importa para CUDA

Esta é, sem exagero, uma das conexões mais diretas e importantes de todo o módulo com o que vem na Phase 2 e 3. A forma como GPU esconde a latência ruim de acesso à memória global (centenas de ciclos, pior que CPU) **não é** prefetch de hardware sofisticado — é ter **milhares de threads**, organizadas em warps, com muitas requisições de memória independentes pendentes ao mesmo tempo. Quando um warp fica esperando um load, o Streaming Multiprocessor troca imediatamente pra outro warp que já está pronto pra trabalhar — a GPU inteira é, em espírito, uma máquina de **maximizar MLP através de paralelismo massivo de threads**, em vez de depender de um prefetcher inteligente adivinhando padrões, como a CPU faz.

É exatamente por isso que o **CUDA Best Practices Guide** da NVIDIA enfatiza que baixa **occupancy** (poucos warps disponíveis pra troca) prejudica diretamente a capacidade da GPU de esconder latência de memória — menos warps disponíveis significa menos "requisições em voo" possíveis simultaneamente, exatamente o mesmo raciocínio de MLP que você acabou de estudar, só que a unidade que gera paralelismo mudou de "múltiplos acumuladores numa thread" pra "múltiplas threads inteiras, rodando em paralelo".

7. Erros comuns

- Achar que "acesso à memória é sempre lento do mesmo jeito" — a diferença entre dependent e independent load muda o comportamento por ordens de magnitude, não é detalhe fino.
- Escrever um benchmark de latência com padrão previsível (stride fixo, sequencial) achando que está medindo "a pior latência possível" — o prefetcher vai esconder boa parte dela, medindo prefetch, não latência real (o erro que a primeira versão do exemplo 01 cometeu).
- Achar que aumentar MLP é sempre grátis — existe um limite de hardware pra quantas requisições ficam em voo simultaneamente; passar desse limite não ajuda mais.
- Confundir "concorrência necessária" (Little's Law, propriedade do hardware/objetivo de bandwidth) com "concorrência disponível" (MLP, propriedade do código/padrão de acesso) — são dois números diferentes, e o resultado real depende do **menor** dos dois (seção 5).

PREDICT BEFORE RUNNING (conceitual)

```
// Trecho A
for (int i = 0; i < n; i++) {
    soma += arr[i];
}

// Trecho B
int idx = 0;
for (int i = 0; i < n; i++) {
    idx = arr[idx];    // arr[i] guarda o índice do próximo passo
}
```

1. Qual trecho tem acessos independentes (endereço conhecido de antemão), e qual tem acessos dependentes?
2. Em qual dos dois o prefetcher de hardware consegue ajudar de verdade?
3. Se **arr** for grande o suficiente pra nunca caber em cache, qual trecho você espera que rode proporcionalmente mais devagar, mesmo fazendo "o mesmo número" de acessos à memória?

► Explicação